

Multi-tenant cell platform

End-to-end learning guide

How the live website works, how requests travel from the browser through Vercel and Render, and how placement, isolation, moves, and wave releases keep operations at $O(\text{waves})$ instead of $O(N)$.

Console (Vercel): linked GitHub repo [vinothhacks/multi-tenant-cell-platform](https://github.com/vinothhacks/multi-tenant-cell-platform), app in frontend/

API (Render): <https://cell-control-plane.onrender.com>

Source: <https://github.com/vinothhacks/multi-tenant-cell-platform>

Local: console <http://localhost:3000> → API <http://localhost:8000>

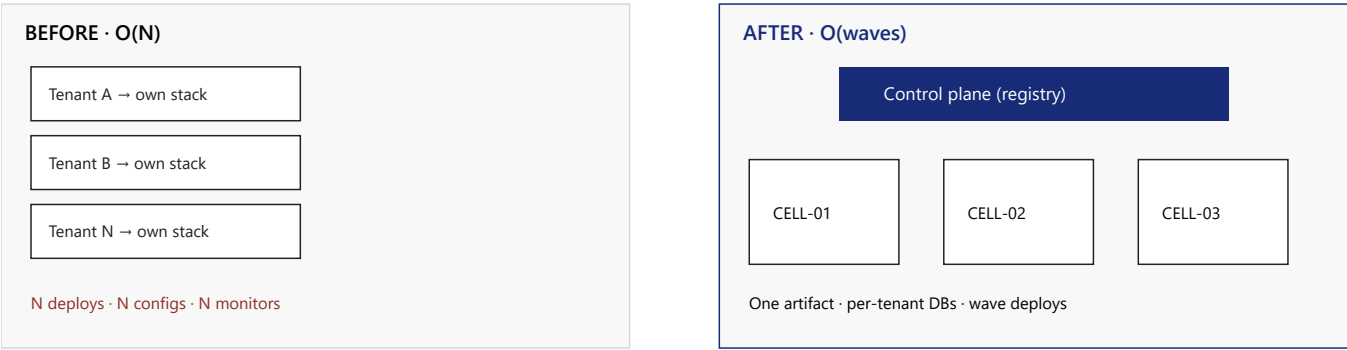
Contents

1. The job: isolation without $O(N)$ operations	Diagram A
2. How the live website is hosted	Diagram B
3. What happens when you click in the console	Diagram C
4. Repository map and the six screens	Diagram D
5. Placement policy (C / D / E are outputs)	Diagram E
6. Tenant lifecycle and the controller	Diagram F
7. Data-plane isolation	Diagram G
8. Tenant move (dump / restore + registry commit)	Diagram H
9. Wave releases	Diagram I
10. Chaos and I1-I10	
11. Run, test, demo video	

1. The job

Isolation is a **security property** (tenant A cannot read, write, or starve tenant B). Separate environments are an **operational topology** (N stacks). The silo design bought isolation by paying $O(N)$ operations. This platform keeps per-tenant databases and makes fleet work scale as waves.

DIAGRAM A — Before vs after



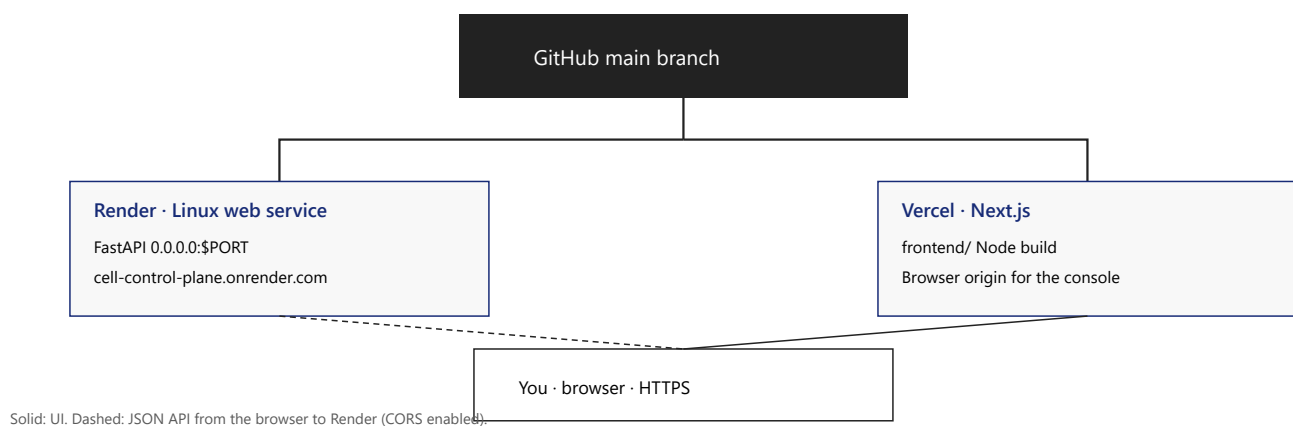
MUST BE TRUE	HOW THIS REPO PROVES IT
No cross-tenant data path	GET /v1/isolation/run plus Django connection manager
Fleet ops are $O(\text{waves})$	One signed release, canary → wave → fleet

Onboarding is a registry write	POST /v1/tenants walks requested → active
Tenants can move	Validated dump/restore, then registry commit
Faults stay contained	Chaos: one DB down, others healthy

2. How the live website is hosted

Three systems, one GitHub repo. Push to `main` updates the sources of truth. Render rebuilds the API. Vercel rebuilds the console. Neither stores tenant data on an ephemeral disk as the product of record — the demo registry is in-memory on the control plane (resets on sleep/restart on the free plan).

DIAGRAM B — GitHub · Render · Vercel

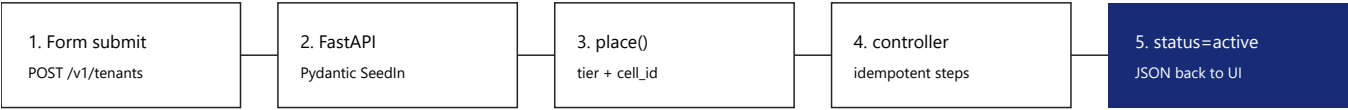


Important Render constraint: the filesystem is ephemeral. Local SQLite files vanish on restart. The showcase control plane keeps the registry in memory so a free web service can still demonstrate placement, moves, and waves. Production would persist the registry in Postgres and tenant DBs on dedicated storage.

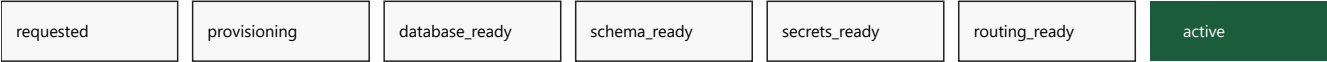
3. What happens when you click in the console

The Next.js app never talks to a database. Every screen is a thin client over `frontend/lib/api.js`. On localhost it uses `http://localhost:8000`. On Vercel (hostname not localhost) it uses `https://cell-control-plane.onrender.com` unless `NEXT_PUBLIC_API_URL` is set.

DIAGRAM C — Example: Create tenant



Controller steps (resume-safe)



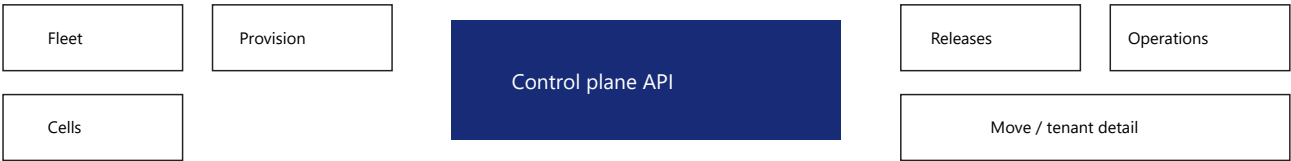
The UI lists those steps as soon as the JSON returns. Humans do not SSH into a cell to create a database.

Other clicks: Seed 20 → POST /v1/synthetic/seed · Start move → POST /v1/tenants/{id}/move · Start V8 → POST /v1/releases · Break DB → POST /v1/chaos/db-down

4. Repository map and the six screens

Repo		Console routes		
PATH	ROLE	#	ROUTE	PROVES
control-plane/	FastAPI registry, placement, controller, moves, releases, isolation, chaos	01	/ Fleet	Registry is source of truth
		02	/ cells	Cell is the blast radius
data-plane/	Django cellapp: tenant middleware, sqlite-per-tenant manager, HMAC Celery envelope	03	/tenants/new	Controller to active
		04	/releases	One artifact, waves
frontend/	Next.js 14 JavaScript console	05	/operations	I1–I10, chaos, cost model
tests/	Phase-gated pytest	—	/tenants/{id}/move	Dump/restore cutover
render.yaml	Blueprint: control-plane + data-plane on 0.0.0.0:\$PORT			
scripts/demo-video/	Playwright clips + TTS + ffmpeg mux			

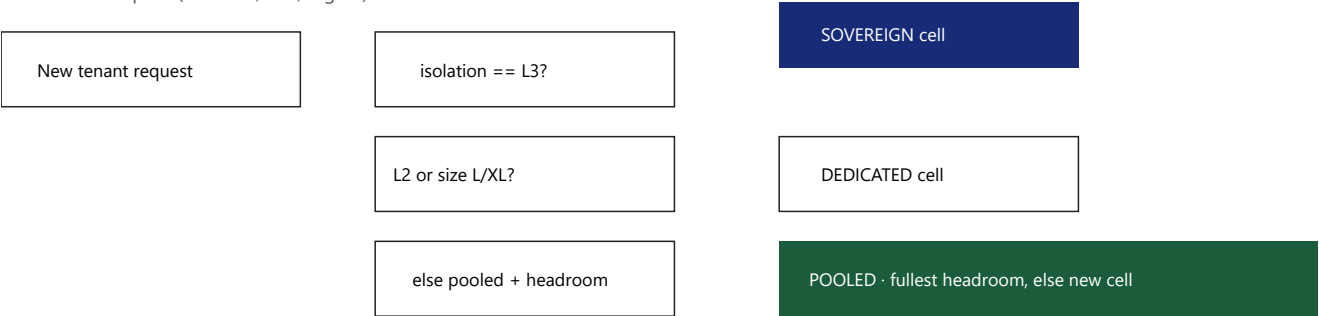
DIAGRAM D — Console is a control-plane skin



5. Placement policy

The policy emits a **tier** (pooled / dedicated / sovereign). It does not “select CELL-01” as a slogan. Pooled capacity is **8**. Overflow creates the next pooled cell (CELL-03 in the demo after CELL-01 and CELL-02 fill).

DIAGRAM E — place(isolation, size, region)



6. Tenant lifecycle

The controller is idempotent. If it crashes after `database_ready`, a retry continues from recorded `completed_steps`. Suspend and resume are registry transitions. Delete is a tombstone, not a silent drop.

DIAGRAM F — Runtime states after active



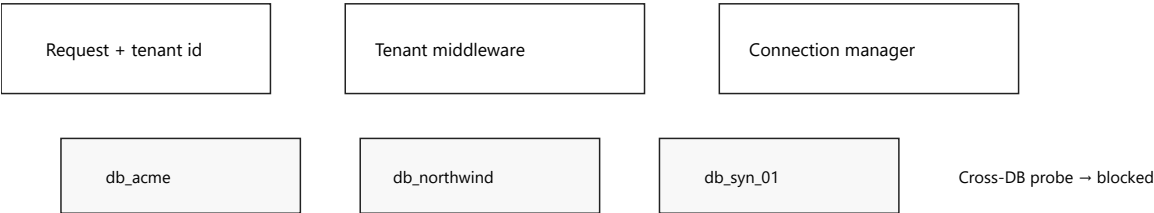
Chaos sets `db_unavailable` → degraded. Other tenants stay active.

7. Data-plane isolation

Django `cellapp` resolves the tenant from the request, then opens that tenant’s database only. Celery tasks carry an HMAC envelope so a worker cannot run as the wrong tenant. This is L1 isolation: own

database, not own Kubernetes cluster.

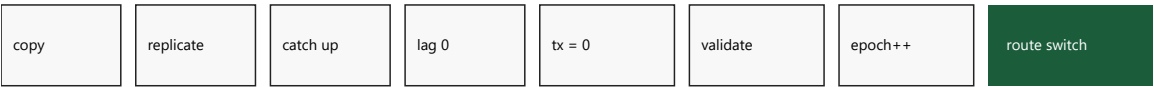
DIAGRAM G — One request, one database



8. Tenant move

Managed Render Postgres does not give you logical replication between arbitrary tenant databases in this showcase. The move path is **validated dump/restore**, then a **registry commit** (epoch++, routing switch). Abort leaves the source authoritative.

DIAGRAM H — Cutover gates



The UI button is “Start move”. The page prints every gate. “Cutover complete” means the registry commit landed.

9. Wave releases

There is one signed image (digest in the UI). Never N per-tenant deploys. Canary a cell, bake, then the next wave. A bad tenant becomes degraded and is isolated from the wave, not used as a reason to roll the whole fleet by hand.

DIAGRAM I — Start V8 then Continue wave



10. Chaos and I1–I10

Operations → **Run I1–I10** posts two live tenant ids and prints PASS/FAIL. **Break one tenant DB** marks one tenant degraded and returns others_healthy: true. That is the blast-radius demo: one partition fails, the cell and the fleet do not.

ID	INTENT (SHORT)
I1–I2	No request or connection uses another tenant's database
I3–I5	Credentials, cache, and queue payloads are tenant-scoped
I6–I8	Backups, restores, and deletes cannot leak across tenants
I9–I10	Noisy neighbour and fault containment stay inside a cell / tenant

11. Run, test, demo video

Local API

```
python -m pip install -r requirements.txt python -m pytest -q python -m uvicorn  
app.main:app --app-dir control-plane --host 0.0.0.0 --port 8000
```

Local console

```
cd frontend npm install npm run dev
```

Explaining video (voice first, then silent Playwright motion — they never overlap)

```
cd scripts/demo-video npm install npx playwright install chromium python generate_audio.py  
node record_scenes.js python mux.py # output/cell-platform-demo.mp4 – not committed; attach  
as a GitHub Release
```

Playwright MCP is optional. This repo records with the Playwright library so the demo is reproducible in CI-like local runs. Do not commit `OPENROUTER_API_KEY`. Neural TTS uses Edge voices when that key is absent.

Deeper architecture, ADRs, threat model, and phase gates: [docs/architecture.md](#), [docs/decisions.md](#), [docs/threat-model.md](#), [docs/phases/](#).